VisionQ

Documento de validación y pruebas

Pruebas funcionales, casos extremos, validación del comportamiento de los modelos, rendimiento, resultados y conclusiones

Asignatura: Gestión de proyectos de inteligencia artificial — Máster en Inteligencia Artificial

Actividad: Fase II. Ejecución — solución técnica completa y desplegable

Profesora: Tania Mendoza

Estudiante: Rene

Repositorio: https://github.com/edgdark80-lgtm/VisionQ

Versión: 2.0.0

Fecha: 29 de octubre de 2026

Contenido

1. Objetivo y alcance

Este documento presenta la validación de la solución desplegable **VisionQ v2.0.0**. El objetivo es demostrar, con evidencia reproducible, que (a) el contenedor Docker se construye y ejecuta sin errores, (b) el frontend y el backend se comunican correctamente a través de la API, (c) la API responde de forma correcta y segura ante entradas válidas, inválidas y extremas, y (d) los modelos de IA mantienen un comportamiento aceptable y predecible fuera de las condiciones ideales de laboratorio.

Se validan los dos módulos de la solución: la **inspección visual de defectos** (clasificador de 6 clases entrenado con NEU-DET; Song & Yan, 2013) y la **predicción de fallas de máquina** (Random Forest entrenado con AI4I 2020; Matzka, 2020). Toda la evidencia cruda (registros, respuestas JSON, gráficas y capturas) se encuentra en `docs/evidence/` del repositorio.

2. Entorno y estrategia de pruebas

Tabla 1. *Entornos en los que se ejecutaron las pruebas*

Entorno	Detalle	Pruebas ejecutadas
Local – Python	Python 3.11.18, Linux x86_64, dependencias de `requirements-dev.txt`	pytest (46), evaluación de modelos
Local – Docker	Docker Engine 29.9, imagen `visionq:2.0.0`, `--read-only --tmpfs /tmp`	Build sin caché, healthcheck, smoke E2E (11), carga, capturas UI
Google Colab (simulado)	Cuaderno `VisionQ_Colab.ipynb` ejecutado de inicio a fin con nbconvert sobre un clon limpio, Python 3.13	pytest (46), integración API, casos extremos, smoke, carga, evaluación completa
CI (GitHub Actions)	`ubuntu-latest`, Python 3.11	Lint, pytest, build, smoke E2E en cada push

La estrategia sigue una pirámide de pruebas adaptada a sistemas de aprendizaje automático (Breck et al., 2017), en la que además de probar el software se prueban los datos y el modelo:

Tabla 2. *Niveles de prueba*

Nivel	Herramienta	Qué verifica	Cantidad
Funcional / integración	pytest + `TestClient` (`tests/test_api_functional.py`)	Contrato de cada endpoint, frontend servido, encabezados, métricas	10
Casos extremos	pytest (`tests/test_edge_cases.py`)	Entradas vacías, corruptas, fuera de rango, lotes límite, seguridad	32
Calidad del modelo	pytest (`tests/test_model_quality.py`)	Umbrales mínimos de métricas y *hold-out* externo	4
E2E sobre contenedor	`scripts/smoke_test.sh`	Flujo completo HTTP contra la imagen Docker	11
Rendimiento	`scripts/load_test.py`	Latencia p50/p95/p99, rendimiento y errores bajo concurrencia	6 escenarios
Robustez del modelo	`scripts/evaluate_models.py`	Perturbaciones, entradas fuera de distribución, umbrales	15 + 10 + barridos

Las 46 pruebas automatizadas (10 funcionales, 32 de casos extremos contando las parametrizaciones, 4 de calidad) **pasaron en su totalidad** en los entornos local y Colab:

```
$ pytest -q
..............................................                           [100%]
46 passed in 14.59s
```

3. Validación de la contenerización

Tabla 3. *Resultados de la validación del contenedor*

Prueba	Resultado esperado	Resultado obtenido	Estado
Build sin caché	Imagen construida sin errores	`visionq:2.0.0` en 1 min 25 s; 212 MB comprimida	Aprobada
Arranque	Contenedor en ejecución	`Up` en < 2 s	Aprobada
Healthcheck de Docker	Estado `healthy`	`healthy` a los ~10 s	Aprobada
Solo lectura y no root	Funciona con `--read-only`, UID 1000	Todas las rutas operan; sin escrituras en disco	Aprobada
Prueba de humo E2E	11/11 verificaciones	11 OK, 0 fallas	Aprobada
Frontend servido por el contenedor	`/ui/` HTTP 200 y consumo de la API	Verificado con Playwright (8 capturas)	Aprobada
Mismo código en Colab	Mismos resultados que el contenedor	Exactitud externa 96.7 % en ambos; métricas idénticas	Aprobada

4. Pruebas funcionales de integración

Las pruebas funcionales verifican el contrato entre frontend y backend: que cada endpoint responde con el código HTTP, la estructura y los campos que el frontend espera.

Tabla 4. *Pruebas funcionales (pytest y prueba de humo)*

ID	Prueba	Verificación	Estado
F-01	Salud del servicio	`GET /health` → 200, `status: ok`, ambos modelos cargados	Aprobada
F-02	Metadatos de modelos	`GET /models` expone clases, métricas, umbrales y fecha de entrenamiento	Aprobada
F-03	Contrato OpenAPI	`/openapi.json` lista los 7 endpoints de negocio	Aprobada
F-04	Frontend servido	`GET /ui/` → 200 con el HTML de la aplicación	Aprobada
F-05	Trazabilidad	`X-Request-ID` enviado por el cliente se propaga en la respuesta	Aprobada
F-06	Inspección individual	`POST /inspect` devuelve clase, confianza, top-3, decisión, OOD, calidad y latencia	Aprobada
F-07	Inspección por lote	`POST /inspect/batch` procesa varias imágenes en una sola llamada	Aprobada
F-08	Falla: normal vs. riesgo	Lectura nominal → `OPERACION_NORMAL`; lectura de riesgo → `FALLA_PROBABLE` con indicadores	Aprobada
F-09	Falla por lote	`POST /failure/predict/batch` con conteo de fallas predichas	Aprobada
F-10	Métricas operativas	`GET /metrics` incrementa contadores por ruta y decisión	Aprobada

Adicionalmente, desde el cuaderno de Colab se enviaron las 30 imágenes de validación externa de NEU-DET (no usadas en el entrenamiento) al endpoint `/api/v1/inspect`: **29 de 30 correctas (96.7 %)**, con latencia media de 43 ms por imagen.



Figura 1. *Evidencia de integración: el frontend muestra el resultado devuelto por la API para una imagen de rayaduras.*

5. Casos extremos evaluados

Los casos extremos se diseñaron para cubrir cuatro riesgos: entradas malformadas que podrían provocar errores internos, valores físicamente imposibles, abuso de recursos y entradas que el modelo nunca vio y sobre las que no debería decidir automáticamente.

5.1 Inspección visual

Tabla 5. *Casos extremos del módulo de inspección*

ID	Caso	Esperado	Obtenido	Estado
E-01	Archivo vacío	Error controlado	HTTP 400 «Archivo vacío»	Aprobada
E-02	Bytes corruptos con MIME de imagen	Error controlado	HTTP 400 «no es una imagen válida o está corrupto»	Aprobada
E-03	MIME no soportado (`text/plain`)	Rechazo	HTTP 415 con tipos permitidos	Aprobada
E-04	Archivo mayor a 5 MB	Rechazo	HTTP 413	Aprobada
E-05	Imagen diminuta (8×8)	No decidir	`REVISION_MANUAL` · `RESOLUCION_INSUFICIENTE`	Aprobada
E-06	Imagen completamente negra / blanca	No decidir	`REVISION_MANUAL` · sub/sobreexpuesta, sin textura	Aprobada
E-07	Ruido aleatorio 200×200	No decidir	`REVISION_MANUAL` · fuera de distribución	Aprobada
E-08	Imagen que no es acero (patrones sintéticos)	No decidir	`REVISION_MANUAL` · fuera de distribución	Aprobada
E-09	Muestra real en RGBA 1600×1600 y RGB 1200×900	Aceptar y convertir	HTTP 200, clase correcta (`scratches`, 0.84)	Aprobada
E-10	Misma imagen en JPG y PNG	Misma predicción	Clase idéntica, diferencia de confianza < 0.02	Aprobada
E-11	Determinismo (misma imagen 2 veces)	Respuesta idéntica	Idéntica	Aprobada
E-12	Lote de 17 imágenes (límite 16)	Rechazo	HTTP 413	Aprobada
E-13	Lote con un archivo inválido	Procesar los válidos	Resultados + lista `errors` por archivo	Aprobada
E-14	Petición sin campo `file`	Rechazo	HTTP 422	Aprobada
E-15	Ruido RGB de alta resolución (1200×900)	No decidir	`RETRABAJO` con confianza 0.84 (no detectado como OOD)	**Limitación**

5.2 Predicción de fallas

Tabla 6. *Casos extremos del módulo de predicción de fallas*

ID	Caso	Esperado	Obtenido	Estado
E-16	Temperatura del aire 200 K o 400 K	Rechazo por rango	HTTP 422 con el campo señalado	Aprobada
E-17	Velocidad negativa, par 500 Nm,	Rechazo por	HTTP 422	Aprobada

ID	Caso	Esperado	Obtenido	Estado
	desgaste negativo	rango		
E-18	Tipo de producto `X` o en minúscula	Rechazo	HTTP 422 «Input should be 'L', 'M' or 'H'»	Aprobada
E-19	Valor no numérico (`torque_nm: "abc"`)	Rechazo	HTTP 422	Aprobada
E-20	Campo faltante	Rechazo	HTTP 422 «Field required»	Aprobada
E-21	Temperatura de proceso < aire (incoherencia física)	Rechazo	HTTP 422 con regla física explicada	Aprobada
E-22	Valores válidos fuera del rango de entrenamiento (par 95 Nm)	Predecir y advertir	HTTP 200 + `out_of_training_range: [torque_nm]`	Aprobada
E-23	Valores en los límites exactos permitidos	Aceptar	HTTP 200	Aprobada
E-24	Monotonía: mayor desgaste de herramienta	Probabilidad no decreciente	Se cumple en el barrido 0–240 min	Aprobada
E-25	Lote vacío / lote de 1001 lecturas	Rechazo	HTTP 422 / HTTP 413	Aprobada
E-26	JSON malformado	Rechazo	HTTP 422 «Datos de entrada inválidos»	Aprobada

5.3 Seguridad y protocolo

Tabla 7. *Casos de seguridad y protocolo*

ID	Caso	Obtenido	Estado
E-27	`VISIONQ_API_KEY` definida y petición sin `X-API-Key`	HTTP 401	Aprobada
E-28	Misma petición con la clave correcta	HTTP 200	Aprobada
E-29	Método no permitido (`GET /inspect`)	HTTP 405	Aprobada
E-30	Errores internos	Mensaje genérico con `request_id`, sin traza expuesta	Aprobada

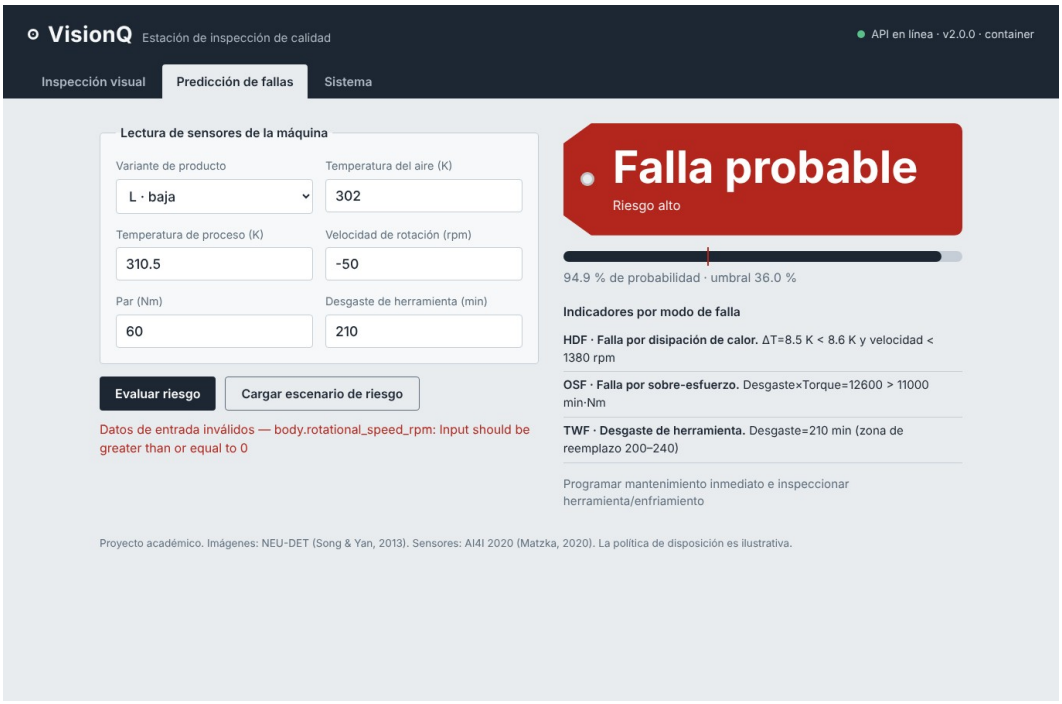


Figura 2. *El frontend muestra el error de validación devuelto por la API (HTTP 422) ante una velocidad negativa.*

6. Validación del modelo de inspección visual

6.1 Desempeño en condiciones nominales

Tabla 8. *Métricas del clasificador de defectos (v2)*

Conjunto	n	Exactitud	F1-macro	Revisión manual	Confianza media
Validación cruzada (5 pliegues, entrenamiento aumentado)	1 416 × 5	—	0.917	—	—
Prueba interna estratificada	354	97.7 %	0.977	0.6 %	0.873
Validación externa (no vista)	30	96.7 %	—	0.0 %	0.868

El único error en la validación externa fue una superficie picada (`pitted_surface_175`) clasificada como `patches` con confianza 0.88; ambas clases comparten manchas oscuras de baja frecuencia, la confusión más habitual reportada para NEU-DET.

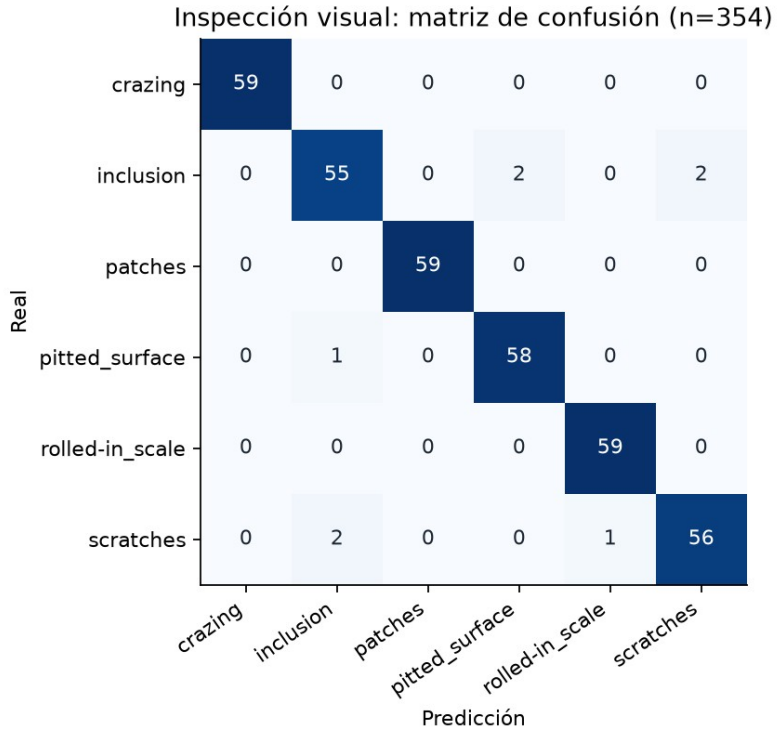


Figura 3. *Matriz de confusión en la prueba interna (n = 354).*

6.2 Robustez ante perturbaciones de captura

En una línea real, la cámara sufre variaciones de iluminación, enfoque, ruido y compresión. Se aplicaron 14 perturbaciones a las 354 imágenes de prueba. La primera versión del modelo (v1, sin aumento de datos) mostró una fragilidad severa —por ejemplo, 27 % de exactitud con desenfoque leve y 32 % con compresión JPEG—, lo que motivó reentrenar con **aumento de datos** (ruido, desenfoque, brillo, rotaciones, reescalado y compresión; 4 variantes por imagen). La tabla compara ambas versiones:

Tabla 9. *Exactitud por perturbación: v1 frente a v2 (n = 354)*

Perturbación	Exactitud v1	Exactitud v2	Revisión manual v2	OOD v2	Confianza v2
Original	98.0 %	97.7 %	0.6 %	0.0 %	0.873
Ruido gaussiano σ=5	63.6 %	96.0 %	0.3 %	0.0 %	0.875
Ruido gaussiano σ=15	46.1 %	94.6 %	0.9 %	0.0 %	0.878
Ruido gaussiano σ=30	31.4 %	82.5 %	2.5 %	0.0 %	0.874
Desenfoque r=1	44.6 %	96.0 %	0.9 %	0.3 %	0.871
Desenfoque r=2.5	27.1 %	92.4 %	4.5 %	3.1 %	0.861
Brillo +30 %	86.7 %	88.4 %	9.3 %	6.2 %	0.850
Brillo −30 %	97.2 %	95.2 %	0.3 %	0.0 %	0.867
Contraste −50 %	93.8 %	78.3 %	2.3 %	0.0 %	0.853
Rotación 90°	65.0 %	91.0 %	1.7 %	0.9 %	0.873
Rotación 180°	93.8 %	95.5 %	1.1 %	0.3 %	0.869
Espejo horizontal	94.6 %	96.9 %	1.1 %	0.3 %	0.868
Reescalado 50 %	36.4 %	96.3 %	1.7 %	0.6 %	0.870
Recorte central 70 %	38.4 %	96.0 %	1.7 %	0.3 %	0.869
JPEG calidad 20	32.5 %	96.6 %	2.0 %	1.7 %	0.868

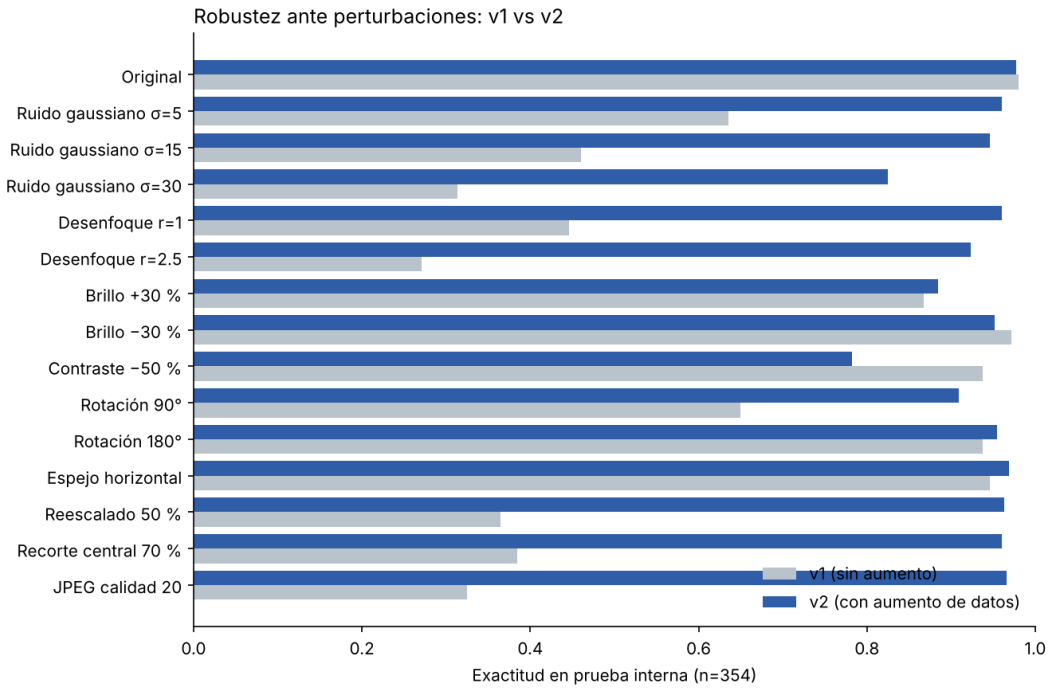


Figura 4. *Comparación de robustez entre v1 y v2: el aumento de datos elimina la fragilidad ante ruido, desenfoque, reescalado, recorte y compresión.*

Análisis. v2 mantiene una exactitud ≥ 91 % en 12 de las 14 perturbaciones, a costa de una caída de 0.3 puntos en condiciones ideales (98.0 % → 97.7 %). Persisten dos debilidades: **contraste reducido al 50 %**

(78.3 %, peor que v1, porque el aumento no incluyó cambios de contraste) y **ruido gaussiano intenso σ = 30** (82.5 %). Con brillo +30 % la exactitud baja a 88.4 %, pero el sistema envía el 9.3 % de las imágenes a revisión manual y la exactitud sobre las decisiones automáticas se mantiene en 94.7 %: el mecanismo de supervisión humana funciona como red de seguridad.

6.3 Entradas fuera de distribución

Un clasificador siempre devuelve alguna clase, incluso ante una imagen que no es acero. Por ello VisionQ combina tres salvaguardas: control de calidad de captura, un detector de novedad sobre el espacio de características (puntuación de un *Isolation Forest* y distancia z máxima) y el umbral de confianza.

Tabla 10. *Respuesta del sistema ante entradas que no son acero*

Entrada sintética	Decisión	OOD	Alertas de calidad
Ruido uniforme	REVISION_MANUAL	Sí	—
Ruido gaussiano puro	REVISION_MANUAL	Sí	—
Tablero de ajedrez	REVISION_MANUAL	Sí	—
Gradiente lineal	REVISION_MANUAL	Sí	—
Imagen negra	REVISION_MANUAL	Sí	IMAGEN_SUBEXPUESTA, IMAGEN_UNIFORME_SIN_TEXTURA
Imagen blanca	REVISION_MANUAL	Sí	IMAGEN_SOBREEXPUESTA, IMAGEN_UNIFORME_SIN_TEXTURA
Gris uniforme	REVISION_MANUAL	Sí	IMAGEN_UNIFORME_SIN_TEXTURA
Círculos concéntricos	REVISION_MANUAL	Sí	—
Franjas horizontales	REVISION_MANUAL	Sí	—
Texto/rejilla fina	REVISION_MANUAL	Sí	—

Las 10 entradas fueron derivadas a revisión manual (**tasa de detección 100.0 %**). Conviene observar que la clase predicha subyacente fue siempre «scratches» con confianza cercana a 0.79: sin las salvaguardas, todas habrían generado una disposición automática errónea. Esto confirma que la confianza del clasificador por sí sola no es un indicador fiable de validez de la entrada.

6.4 Selección del umbral de confianza

Tabla 11. *Cobertura automática frente a exactitud según el umbral de confianza*

Umbral	Cobertura (decisión automática)	Exactitud sobre decisiones automáticas
0.50	100.0 %	97.7 %
0.60	100.0 %	97.7 %
0.70	100.0 %	97.7 %
0.80	84.2 %	98.0 %
0.85	71.5 %	98.8 %
0.90	31.9 %	100.0 %
0.95	7.9 %	100.0 %

Con el umbral operativo de 0.60, el 100 % de las imágenes nominales se decide automáticamente con 97.7 % de exactitud. Elevarlo a 0.85 sube la exactitud a 98.8 % pero envía el 29 % de las piezas a revisión manual. Se mantiene 0.60 como valor por defecto, configurable por planta mediante `VISIONQ_CONFIDENCE_THRESHOLD` según el costo relativo de un escape de calidad frente a la carga del inspector.

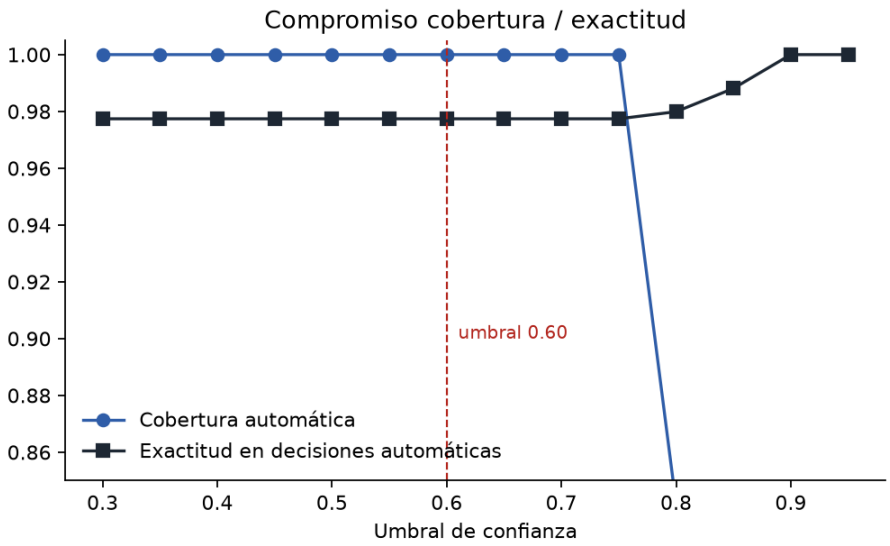


Figura 5. *Curva de cobertura y exactitud según el umbral de confianza.*

7. Validación del modelo de predicción de fallas

7.1 Métricas en el conjunto de prueba

El conjunto AI4I 2020 está fuertemente desbalanceado (3.39 % de fallas), por lo que la exactitud no es informativa. Se reportan ROC-AUC, PR-AUC, precisión, *recall* y F1 sobre 2 000 registros de prueba nunca vistos; el umbral (0.36) se seleccionó en un conjunto de validación separado para maximizar F1, evitando sobreajustarlo al conjunto de prueba.

Tabla 12. *Métricas del modelo de fallas (n = 2 000, 68 fallas)*

ROC-AUC	PR-AUC	Precisión	Recall	F1	VP	FP	FN	VN
0.984	0.875	0.877	0.838	0.857	57	8	11	1 924

Tabla 13. *Efecto del umbral de decisión en el conjunto de prueba*

Umbral	VP	FP	FN	Precisión	Recall
0.20	59	29	9	0.670	0.868
0.30	58	12	10	0.829	0.853
0.36 (elegido)	57	8	11	0.877	0.838
0.50	54	2	14	0.964	0.794
0.70	43	2	25	0.956	0.632

Bajar el umbral a 0.20 detectaría 2 fallas adicionales a cambio de 21 falsas alarmas más; subirlo a 0.50 reduce las falsas alarmas a 2 pero deja pasar 3 fallas más. El umbral es configurable (`VISIONQ_FAILURE_THRESHOLD`) para que mantenimiento ajuste el equilibrio según el costo de un paro no planeado frente al de una inspección innecesaria.

7.2 Comportamiento por modo de falla

Tabla 14. *Recall por modo de falla en el conjunto de prueba*

Modo de falla	Casos	Detectados	Recall
TWF · Desgaste de herramienta	10	2	20.0 %
HDF · Disipación de calor	29	28	96.5 %
PWF · Potencia	13	13	100.0 %
OSF · Sobre-esfuerzo	16	16	100.0 %
RNF · Aleatoria	4	0	0.0 %

El modelo detecta prácticamente todas las fallas con causa física determinista (disipación de calor, potencia y sobre-esfuerzo), pero solo 2 de 10 fallas por desgaste de herramienta (TWF) y ninguna falla aleatoria (RNF). Esto es coherente con la construcción del dataset: TWF ocurre en un instante aleatorio dentro de la ventana de 200–240 min de desgaste y RNF es aleatoria por definición (Matzka, 2020). Para mitigarlo, la API devuelve **indicadores por modo de falla** basados en reglas físicas; por ejemplo, alerta de «zona de reemplazo» cuando el desgaste supera 200 min aunque la probabilidad sea baja.

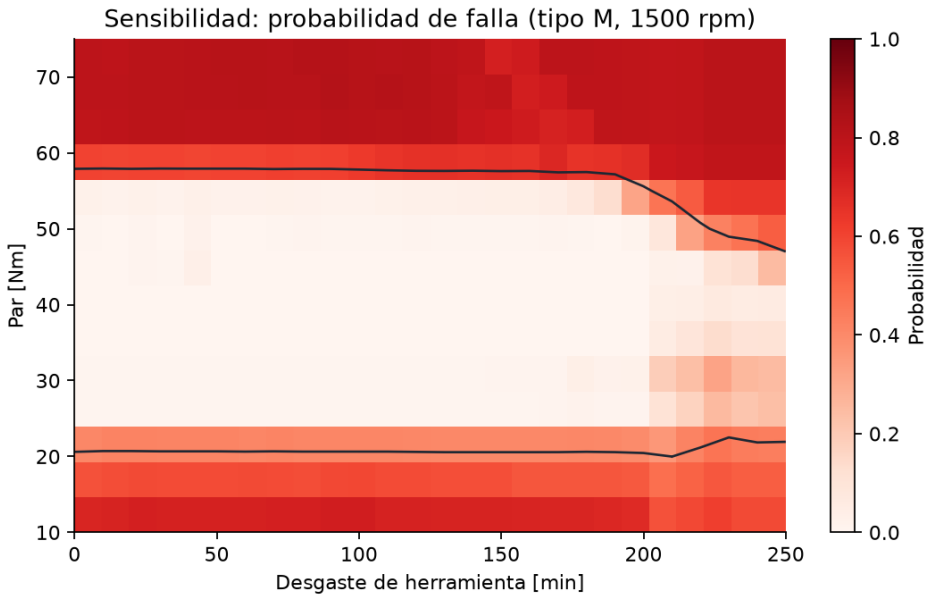


Figura 6. *Sensibilidad de la probabilidad de falla a las variables de operación.*

8. Pruebas de rendimiento

Se ejecutaron 60 peticiones por escenario contra el contenedor (1 vCPU efectiva, un proceso Uvicorn) con concurrencia 1, 4 y 8.

Tabla 15. *Resultados de la prueba de carga contra el contenedor*

Endpoint	Concurrencia	Peticiones	Rendimiento (req/s)	p50 (ms)	p95 (ms)	p99 (ms)	Errores
/inspect	1	60	18.3	56.4	63.5	66	0.0 %
/inspect	4	60	24	163.4	248.3	280.9	0.0 %
/inspect	8	60	22.3	349.7	487.7	622.7	0.0 %
/failure/predict	1	60	39.7	24.2	35.8	37.5	0.0 %
/failure/predict	4	60	23.4	165.9	274.1	320.1	0.0 %
/failure/predict	8	60	22.4	345.6	475.8	536.4	0.0 %

No se registraron errores en ningún escenario. La latencia individual de inferencia es de ~55 ms (inspección) y ~25 ms (fallas). El rendimiento se satura en torno a 22–24 req/s porque la extracción de características es intensiva en CPU: al aumentar la concurrencia crece la latencia en cola, no la tasa de errores. Para una estación de inspección (≈ 1 pieza por segundo) la capacidad es holgada; para varias líneas basta escalar horizontalmente (`--max-instances` en Cloud Run) o aumentar `WEB_CONCURRENCY` en instancias con más núcleos.

9. Hallazgos, limitaciones y acciones

Tabla 16. *Hallazgos de la validación y acciones propuestas*

Hallazgo	Impacto	Acción
v1 era frágil ante ruido, desenfoque y compresión	Alto: fallaría con cámaras reales	**Resuelto** en v2 con aumento de datos (exactitud con desenfoque 27 % → 92 %)
Contraste −50 % reduce la exactitud a 78 %	Medio	Añadir aumento de contraste y una alerta de calidad por contraste bajo en la próxima versión
Ruido RGB de alta resolución no se detecta como OOD (E-15)	Medio: al reducirse a 200×200 el ruido se suaviza y parece textura	Calcular las alertas de calidad sobre la imagen original antes del reescalado; ampliar el detector OOD con entradas sintéticas
Recall de fallas TWF 20 % y RNF 0 %	Medio, inherente al dataset	Mantener los indicadores por reglas físicas; incorporar series temporales de desgaste
Rendimiento limitado por CPU (~23 req/s por instancia)	Bajo para el caso de uso	Escalado horizontal; vectorizar la extracción de características si crece la demanda
Validación externa de visión con solo 30 imágenes	Medio: intervalo de confianza amplio	Recolectar imágenes de la línea real para una validación piloto

10. Conclusiones

La validación demuestra que VisionQ cumple los criterios de la Fase II:

- **Contenerización y despliegue local:** la imagen se construye sin errores en 85 segundos, arranca en estado `healthy` con usuario no root y sistema de archivos de solo lectura, y supera 11/11 verificaciones E2E.
- **Integración técnica:** el frontend consume la API versionada y documentada (OpenAPI), con contratos verificados por 10 pruebas funcionales y evidencia visual automatizada.
- **Pruebas y validación del modelo:** 46/46 pruebas automatizadas aprobadas, 30 casos extremos evaluados con respuesta controlada en 29 de ellos, y un análisis de robustez que detectó y corrigió una debilidad crítica de la primera versión del modelo.
- **Reproducibilidad:** los mismos resultados se obtuvieron en el contenedor, en Python local y en el cuaderno de Colab, lo que confirma la consistencia entre entornos.

El hallazgo más valioso del proceso fue que un modelo con 98 % de exactitud en laboratorio podía caer por debajo del 35 % ante perturbaciones realistas. La evaluación de robustez, más que la métrica nominal, fue la que determinó la versión que se despliega. Asimismo, la combinación de control de calidad de captura, detección fuera de distribución y umbral de confianza resultó indispensable: el clasificador asignó con 79 % de confianza una clase de defecto a imágenes que no eran acero. Las limitaciones remanentes (contraste bajo, ruido de alta resolución, fallas aleatorias) están identificadas, tienen impacto acotado gracias a la revisión manual y constituyen el punto de partida del monitoreo y reentrenamiento continuo de la Fase III.

Referencias

Breck, E., Cai, S., Nielsen, E., Salib, M., & Sculley, D. (2017). The ML test score: A rubric for ML production readiness and technical debt reduction. En *2017 IEEE International Conference on Big Data* (pp. 1123–1132). IEEE. https://doi.org/10.1109/BigData.2017.8258038

Hendrycks, D., & Dietterich, T. (2019). Benchmarking neural network robustness to common corruptions and perturbations. En *International Conference on Learning Representations*. https://arxiv.org/abs/1903.12261

Liu, F. T., Ting, K. M., & Zhou, Z.-H. (2008). Isolation forest. En *2008 Eighth IEEE International Conference on Data Mining* (pp. 413–422). IEEE. https://doi.org/10.1109/ICDM.2008.17

Matzka, S. (2020). Explainable artificial intelligence for predictive maintenance applications. En *2020 Third International Conference on Artificial Intelligence for Industries (AI4I)* (pp. 69–74). IEEE. https://doi.org/10.1109/AI4I49448.2020.00023

Saito, T., & Rehmsmeier, M. (2015). The precision-recall plot is more informative than the ROC plot when evaluating binary classifiers on imbalanced datasets. *PLOS ONE*, *10*(3), e0118432. https://doi.org/10.1371/journal.pone.0118432

Song, K., & Yan, Y. (2013). A noise robust method based on completed local binary patterns for hot-rolled steel strip surface defects. *Applied Surface Science*, *285*, 858–864. https://doi.org/10.1016/j.apsusc.2013.09.002